

Unsupervised Learning

Unsupervised learning adalah jenis machine learning di mana model diberi tugas untuk mengeksplorasi dan memahami struktur data tanpa memiliki label yang terkait. Salah satu teknik yang sering digunakan dalam unsupervised learning adalah clustering, di mana data dikelompokkan menjadi subset yang memiliki kesamaan tertentu berdasarkan pola atau karakteristik yang ditemukan secara otomatis oleh algoritma, tanpa supervisi eksternal. Clustering bertujuan untuk mengidentifikasi hubungan intrinsik antara data, memungkinkan pemahaman yang lebih dalam tentang pola dan struktur yang tersembunyi dalam data, serta membantu dalam pengelompokan data yang serupa ke dalam kategori yang berbeda.

Soal 1 : Dataset preparation

- Download dataset : <https://www.kaggle.com/datasets/arjunbhasin2013/ccdata>
(<https://www.kaggle.com/datasets/arjunbhasin2013/ccdata>)
- Upload di google drive atau github masing-masing
- Jelaskan tujuan penggunaan dataset ini
- Jelaskan feature dataset ini

read dataset

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/CC%20GENERAL.csv', de
df.head()
#df.describe()
```

```
Out[1]:
```

	CUST_ID	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE
0	C10001	40.900749	0.818182	95.40	0.00	95.4	0.000000
1	C10002	3202.467416	0.909091	0.00	0.00	0.0	6442.945483
2	C10003	2495.148862	1.000000	773.17	773.17	0.0	0.000000
3	C10004	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017
4	C10005	817.714335	1.000000	16.00	16.00	0.0	0.000000

dataset description

```
In [ ]: df.shape
```

```
Out[2]: (8950, 18)
```

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 8950 entries, 0 to 8949
```

```
Data columns (total 18 columns):
```

#	Column	Non-Null Count	Dtype
0	CUST_ID	8950 non-null	object
1	BALANCE	8950 non-null	float64
2	BALANCE_FREQUENCY	8950 non-null	float64
3	PURCHASES	8950 non-null	float64
4	ONEOFF_PURCHASES	8950 non-null	float64
5	INSTALLMENTS_PURCHASES	8950 non-null	float64
6	CASH_ADVANCE	8950 non-null	float64
7	PURCHASES_FREQUENCY	8950 non-null	float64
8	ONEOFF_PURCHASES_FREQUENCY	8950 non-null	float64
9	PURCHASES_INSTALLMENTS_FREQUENCY	8950 non-null	float64
10	CASH_ADVANCE_FREQUENCY	8950 non-null	float64
11	CASH_ADVANCE_TRX	8950 non-null	int64
12	PURCHASES_TRX	8950 non-null	int64
13	CREDIT_LIMIT	8949 non-null	float64
14	PAYMENTS	8950 non-null	float64
15	MINIMUM_PAYMENTS	8637 non-null	float64
16	PRC_FULL_PAYMENT	8950 non-null	float64
17	TENURE	8950 non-null	int64

```
dtypes: float64(14), int64(3), object(1)
```

```
memory usage: 1.2+ MB
```

```
In [ ]: df.describe().T
```

```
Out[4]:
```

	count	mean	std	min	25%	50%	75%	
BALANCE	8950.0	1564.474828	2081.531879	0.000000	128.281915	873.385231	2054.140036	19043.7
BALANCE_FREQUENCY	8950.0	0.877271	0.236904	0.000000	0.888889	1.000000	1.000000	1.0
PURCHASES	8950.0	1003.204834	2136.634782	0.000000	39.635000	361.280000	1110.130000	49039.5
ONEOFF_PURCHASES	8950.0	592.437371	1659.887917	0.000000	0.000000	38.000000	577.405000	40761.2
INSTALLMENTS_PURCHASES	8950.0	411.067645	904.338115	0.000000	0.000000	89.000000	468.637500	22500.0
CASH_ADVANCE	8950.0	978.871112	2097.163877	0.000000	0.000000	0.000000	1113.821139	47137.1
PURCHASES_FREQUENCY	8950.0	0.490351	0.401371	0.000000	0.083333	0.500000	0.916667	1.0
ONEOFF_PURCHASES_FREQUENCY	8950.0	0.202458	0.298336	0.000000	0.000000	0.083333	0.300000	1.0
PURCHASES_INSTALLMENTS_FREQUENCY	8950.0	0.364437	0.397448	0.000000	0.000000	0.166667	0.750000	1.0
CASH_ADVANCE_FREQUENCY	8950.0	0.135144	0.200121	0.000000	0.000000	0.000000	0.222222	1.5
CASH_ADVANCE_TRX	8950.0	3.248827	6.824647	0.000000	0.000000	0.000000	4.000000	123.0
PURCHASES_TRX	8950.0	14.709832	24.857649	0.000000	1.000000	7.000000	17.000000	358.0
CREDIT_LIMIT	8949.0	4494.449450	3638.815725	50.000000	1600.000000	3000.000000	6500.000000	30000.0
PAYMENTS	8950.0	1733.143852	2895.063757	0.000000	383.276166	856.901546	1901.134317	50721.4
MINIMUM_PAYMENTS	8637.0	864.206542	2372.446607	0.019163	169.123707	312.343947	825.485459	76406.2
PRC_FULL_PAYMENT	8950.0	0.153715	0.292499	0.000000	0.000000	0.000000	0.142857	1.0
TENURE	8950.0	11.517318	1.338331	6.000000	12.000000	12.000000	12.000000	12.0

```
In [ ]: df.corr(numeric_only=True)
```

```
Out[8]:
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PU
BALANCE	1.000000	0.322412	0.181261	0.164350	
BALANCE_FREQUENCY	0.322412	1.000000	0.133674	0.104323	
PURCHASES	0.181261	0.133674	1.000000	0.916845	
ONEOFF_PURCHASES	0.164350	0.104323	0.916845	1.000000	
INSTALLMENTS_PURCHASES	0.126469	0.124292	0.679896	0.330622	
CASH_ADVANCE	0.496692	0.099388	-0.051474	-0.031326	
PURCHASES_FREQUENCY	-0.077944	0.229715	0.393017	0.264937	
ONEOFF_PURCHASES_FREQUENCY	0.073166	0.202415	0.498430	0.524891	
PURCHASES_INSTALLMENTS_FREQUENCY	-0.063186	0.176079	0.315567	0.127729	
CASH_ADVANCE_FREQUENCY	0.449218	0.191873	-0.120143	-0.082628	
CASH_ADVANCE_TRX	0.385152	0.141555	-0.067175	-0.046212	
PURCHASES_TRX	0.154338	0.189626	0.689561	0.545523	
CREDIT_LIMIT	0.531283	0.095843	0.356963	0.319724	
PAYMENTS	0.322802	0.065008	0.603264	0.567292	
MINIMUM_PAYMENTS	0.398684	0.132569	0.093860	0.048755	
PRC_FULL_PAYMENT	-0.318959	-0.095082	0.180379	0.132763	
TENURE	0.072692	0.119776	0.086288	0.064150	

preprocessing

```
In [ ]: df_new = df.drop('CUST_ID', axis = 1)
df_new.head()
```

Out[9]:

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES
0	40.900749	0.818182	95.40	0.00	95.4	0.000000	
1	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	
2	2495.148862	1.000000	773.17	773.17	0.0	0.000000	
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	
4	817.714335	1.000000	16.00	16.00	0.0	0.000000	



check missing values

```
In [ ]: df_new.isnull().sum()
```

```
Out[10]: BALANCE                                0
          BALANCE_FREQUENCY                    0
          PURCHASES                            0
          ONEOFF_PURCHASES                     0
          INSTALLMENTS_PURCHASES               0
          CASH_ADVANCE                         0
          PURCHASES_FREQUENCY                  0
          ONEOFF_PURCHASES_FREQUENCY           0
          PURCHASES_INSTALLMENTS_FREQUENCY     0
          CASH_ADVANCE_FREQUENCY              0
          CASH_ADVANCE_TRX                     0
          PURCHASES_TRX                       0
          CREDIT_LIMIT                         1
          PAYMENTS                             0
          MINIMUM_PAYMENTS                    313
          PRC_FULL_PAYMENT                    0
          TENURE                               0
          dtype: int64
```

imputation

```
In [ ]: df_new['MINIMUM_PAYMENTS'].fillna(df_new['MINIMUM_PAYMENTS'].median(), inplace=True)
         df_new['CREDIT_LIMIT'].fillna(df_new['CREDIT_LIMIT'].median(), inplace=True)
```

```
In [ ]: df_new.isnull().sum()
```

```
Out[12]: BALANCE                                0
          BALANCE_FREQUENCY                    0
          PURCHASES                            0
          ONEOFF_PURCHASES                     0
          INSTALLMENTS_PURCHASES               0
          CASH_ADVANCE                          0
          PURCHASES_FREQUENCY                  0
          ONEOFF_PURCHASES_FREQUENCY           0
          PURCHASES_INSTALLMENTS_FREQUENCY     0
          CASH_ADVANCE_FREQUENCY              0
          CASH_ADVANCE_TRX                     0
          PURCHASES_TRX                        0
          CREDIT_LIMIT                         0
          PAYMENTS                             0
          MINIMUM_PAYMENTS                     0
          PRC_FULL_PAYMENT                     0
          TENURE                               0
          dtype: int64
```

scaling

```
In [ ]: from sklearn.preprocessing import StandardScaler
        #from sklearn.preprocessing import MinMaxScaler

        X = df_new.astype(float).values
        scaler = StandardScaler().fit(X)
        X_new= scaler.transform(X)
        X_new
```

```
Out[13]: array([[ -0.73198937, -0.24943448, -0.42489974, ..., -0.3024      ,
                -0.52555097,  0.36067954],
                [ 0.78696085,  0.13432467, -0.46955188, ...,  0.09749953,
                0.2342269 ,  0.36067954],
                [ 0.44713513,  0.51808382, -0.10766823, ..., -0.0932934 ,
                -0.52555097,  0.36067954],
                ...,
                [-0.7403981 , -0.18547673, -0.40196519, ..., -0.32687479,
                0.32919999, -4.12276757],
                [-0.74517423, -0.18547673, -0.46955188, ..., -0.33830497,
                0.32919999, -4.12276757],
                [-0.57257511, -0.88903307,  0.04214581, ..., -0.3243581 ,
                -0.52555097, -4.12276757]])
```

KMeans

```
In [ ]: from sklearn.cluster import KMeans
```

elbow method

Mencari nilai k

```
In [ ]: inertia_list = []
for num_clusters in range(1, 11):
    kmeans_model = KMeans(n_clusters=num_clusters)
    kmeans_model.fit(X_new)
    inertia_list.append(kmeans_model.inertia_)
    print("For n_clusters = {}, inertia value is {}".format(num_clusters, kmeans_model.inertia_))
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning
warnings.warn(
```

```
For n_clusters = 1, inertia value is 152149.99999999994)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning
warnings.warn(
```

```
For n_clusters = 2, inertia value is 127784.71030749619)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning
warnings.warn(
```

```
For n_clusters = 3, inertia value is 111975.0435932566)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning
warnings.warn(
```

```
For n_clusters = 4, inertia value is 99061.93984229013)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning
warnings.warn(
```

```
For n_clusters = 5, inertia value is 91491.11253232477)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning
warnings.warn(
```

```
For n_clusters = 6, inertia value is 84825.07421442692)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(
```

```
For n_clusters = 7, inertia value is 79551.38361416441)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(
```

```
For n_clusters = 8, inertia value is 74467.91793967017)
```

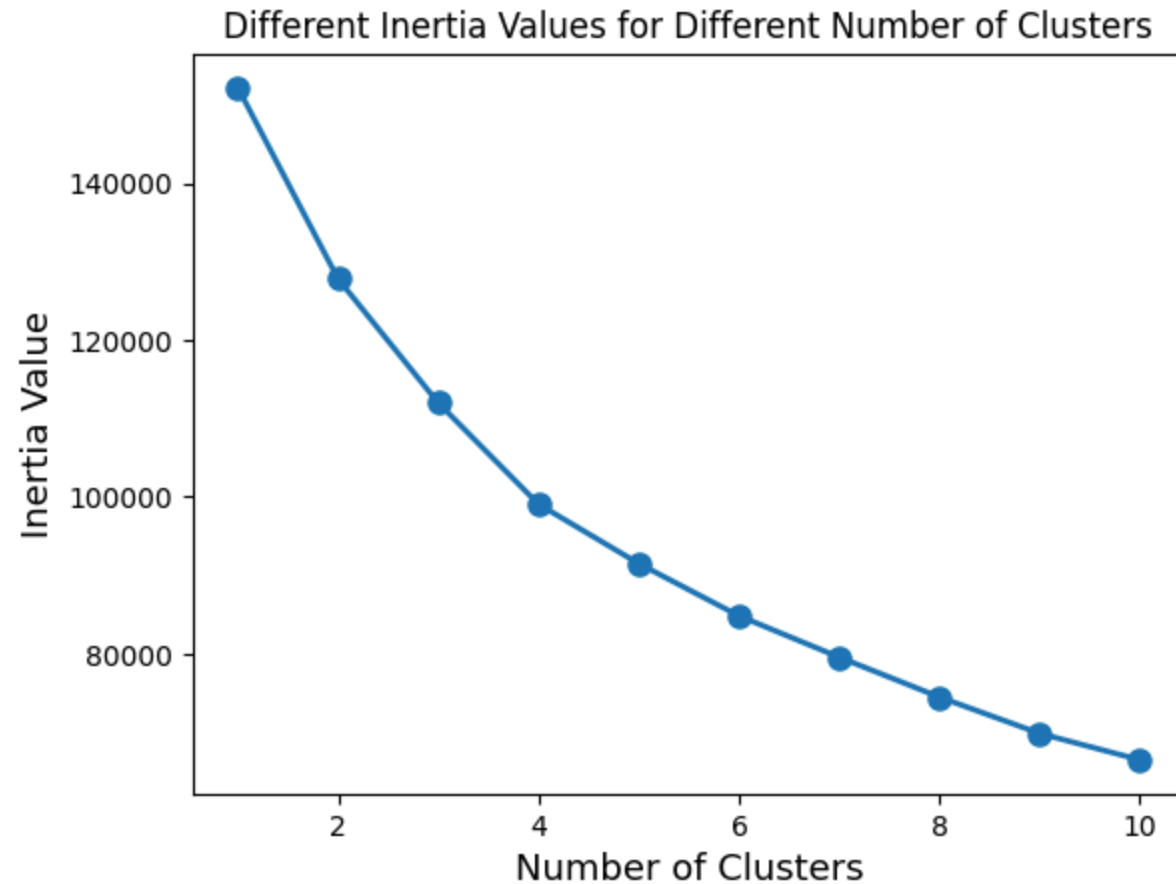
```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(
```

```
For n_clusters = 9, inertia value is 69832.79818134755)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(
```

```
For n_clusters = 10, inertia value is 66459.91548110383)
```

```
In [ ]: plt.plot(range(1,11),inertia_list, marker='o', linewidth=2, markersize=8)
plt.xlabel("Number of Clusters", size=13)
plt.ylabel("Inertia Value", size=13)
plt.title("Inertia values vary depending on the number of clusters utilized.")
plt.show()
```



```
In [ ]: pip install kneed
```

Collecting kneed

Downloading kneed-0.8.5-py3-none-any.whl (10 kB)

Requirement already satisfied: numpy>=1.14.2 in /usr/local/lib/python3.10/dist-packages (from kneed) (1.25.2)

Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from kneed) (1.11.4)

Installing collected packages: kneed

Successfully installed kneed-0.8.5

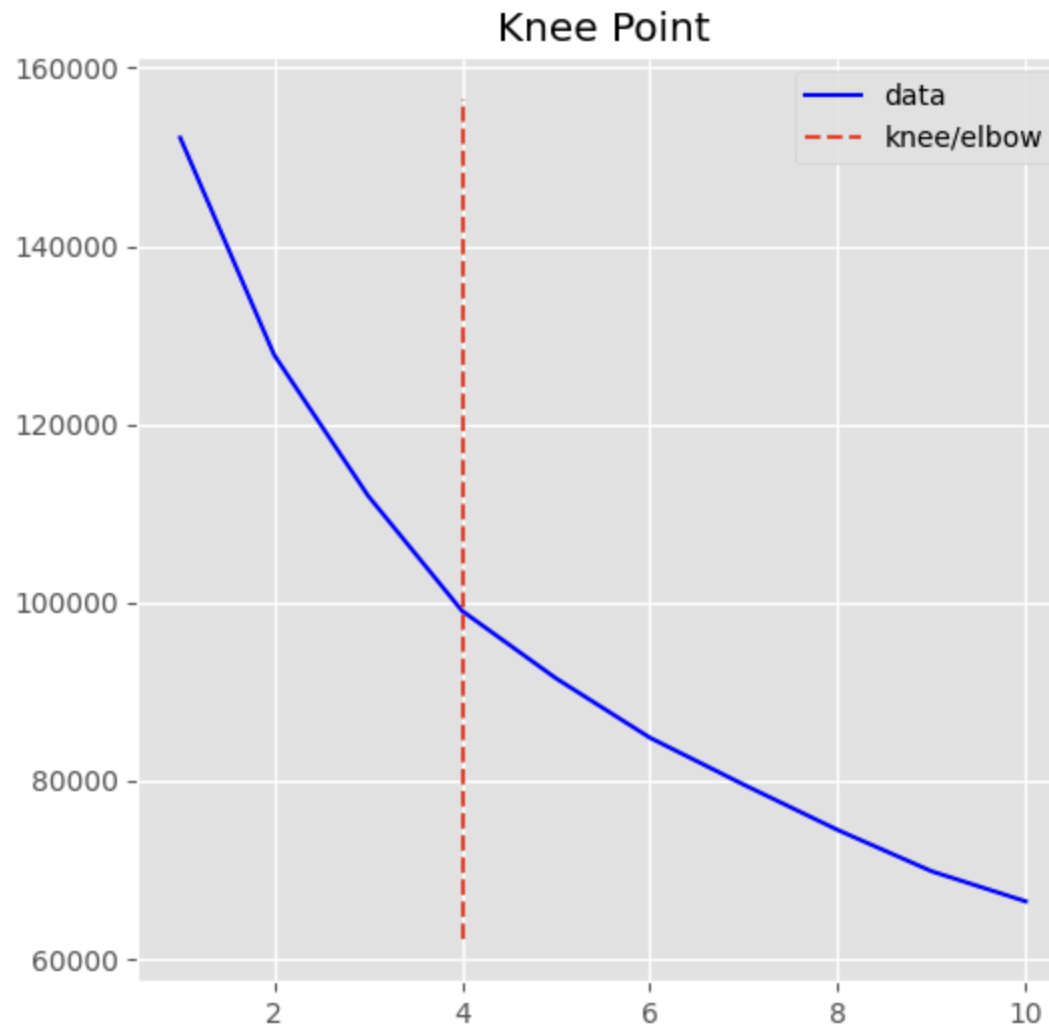
```
In [ ]: from kneed import KneeLocator, DataGenerator
kneedle = KneeLocator(range(1,11),inertia_list, S=1.0, curve='convex', direction='decreasing')
print(round(kneedle.knee, 3))
print(round(kneedle.elbow, 3))
```

4

4

```
In [ ]: plt.style.use('ggplot')
#kneedle.plot_knee_normalized()
kneedle.plot_knee()

# sampai di stage ini kita mendapatkan k=4 yg optimal
# reference
# https://www.kaggle.com/kevinarvai/knee-elbow-point-detection
```



silhouette method

untuk mencari nilai k

```
In [ ]: from sklearn.metrics import silhouette_samples, silhouette_score
```

```
sh_list = []  
for num_clusters in range(2, 11):  
    kmeans = KMeans(n_clusters=num_clusters)  
    cluster_labels = kmeans.fit_predict(X_new)  
  
    score = silhouette_score(X_new, cluster_labels)  
    sh_list.append(score)  
    print("For n_clusters = {}, silhouette score is {}".format(num_clusters, score))
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(  
    For n_clusters = 2, silhouette score is 0.20949692655850133)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(  
    For n_clusters = 3, silhouette score is 0.2504160232402912)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(  
    For n_clusters = 4, silhouette score is 0.1976791965228765)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(  
    For n_clusters = 5, silhouette score is 0.19325195080511473)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(  
    For n_clusters = 6, silhouette score is 0.20273049886535538)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(  
    For n_clusters = 7, silhouette score is 0.20273049886535538)
```

For n_clusters = 7, silhouette score is 0.21498383050560788)

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(
```

For n_clusters = 8, silhouette score is 0.2183690591431352)

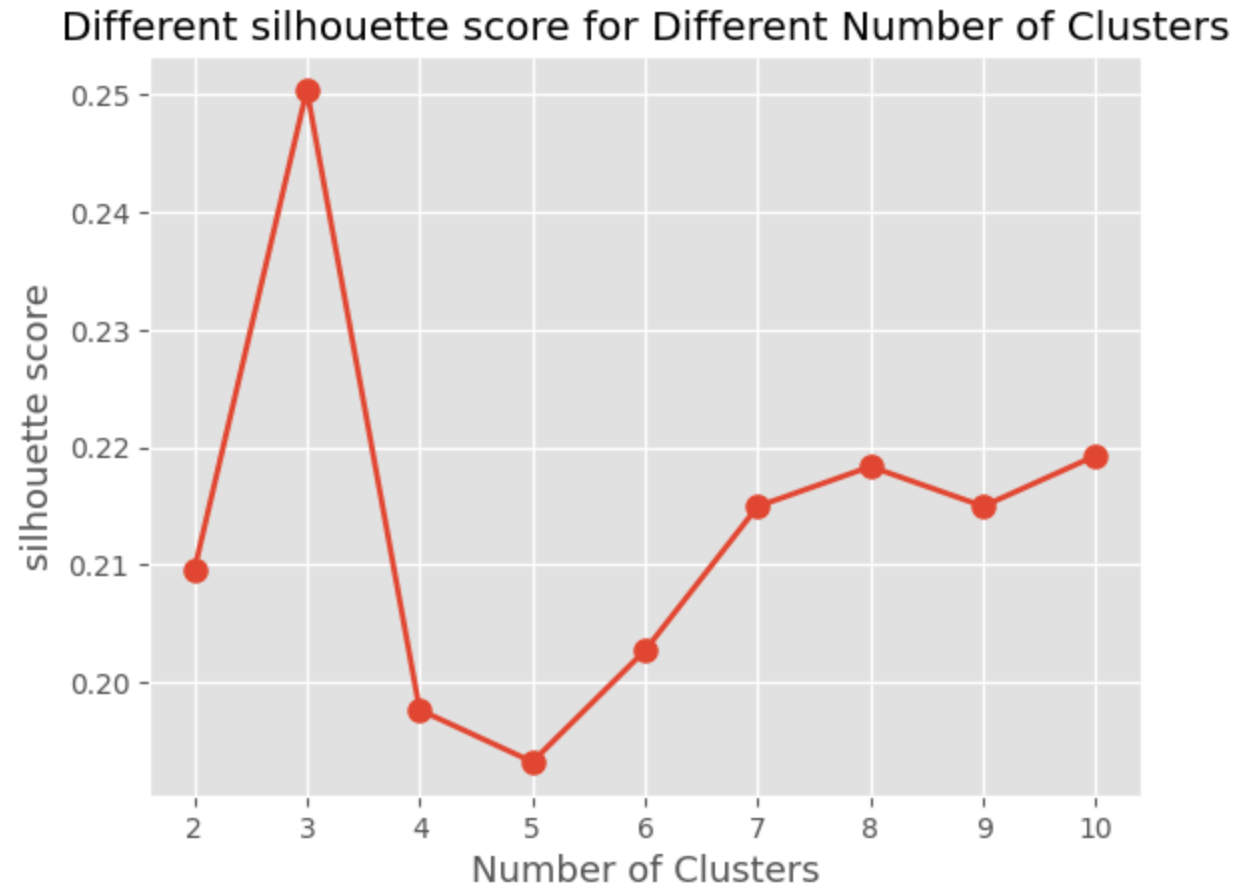
```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(
```

For n_clusters = 9, silhouette score is 0.21499040938936276)

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of  
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning  
warnings.warn(
```

For n_clusters = 10, silhouette score is 0.21930888619601588)

```
In [ ]: plt.plot(range(2,11),sh_list, marker='o', linewidth=2, markersize=8)
plt.xlabel("Number of Clusters", size=13)
plt.ylabel("silhouette score", size=13)
plt.title("Silhouette score values vary depending on the number of clusters utilized.")
plt.show()
# di stage ini kita menemukan k=3 dengan score tertinggi
```



kmeans

```
In [ ]: k_means = KMeans(n_clusters = 3, random_state = 42)
k_means.fit(X_new)
labels = k_means.labels_
df_new['cluster_labels'] = labels
df_new.head()
```

/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of `n\_init` will change from 10 to 'auto' in 1.4. Set the value of `n\_init` explicitly to suppress the warning
warnings.warn(

Out[23]:

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES
0	40.900749	0.818182	95.40	0.00	95.4	0.000000	
1	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	
2	2495.148862	1.000000	773.17	773.17	0.0	0.000000	
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	
4	817.714335	1.000000	16.00	16.00	0.0	0.000000	

```
In [ ]: # check centroids nya
centroids = k_means.cluster_centers_
centroids
```

Out[24]:

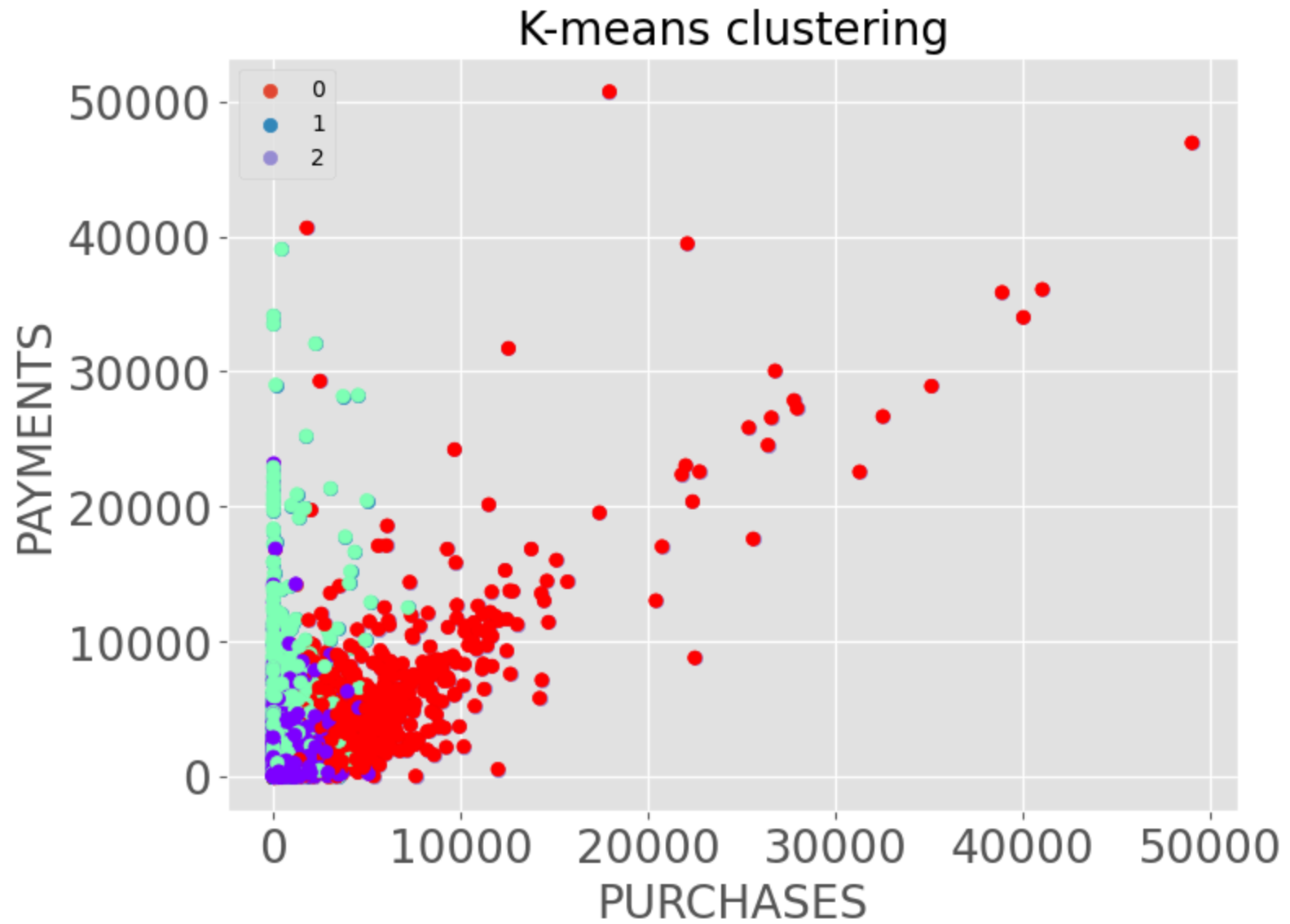
```
array([[ -0.36747893, -0.17788652, -0.23276749, -0.20434258, -0.17486029,
        -0.30907926, -0.06254974, -0.23484457, -0.04810771, -0.33428261,
        -0.29884299, -0.24345751, -0.33560051, -0.28443175, -0.13795689,
         0.00740749, -0.02801772],
       [ 1.16245872,  0.34237903, -0.29035137, -0.2073741 , -0.30561548,
         1.37706384, -0.64418279, -0.30688629, -0.55540367,  1.56021736,
         1.34088368, -0.36748423,  0.59838603,  0.44395469,  0.40373027,
        -0.4100037 , -0.12701831],
       [ 0.32027299,  0.43982886,  1.52988676,  1.28163909,  1.26237969,
        -0.24679312,  1.14289991,  1.56156181,  0.9565123 , -0.35851122,
        -0.2508035 ,  1.68261002,  0.8912358 ,  0.83703323,  0.16246708,
         0.49320477,  0.30315287]])
```

Visualization

```
In [ ]: x1 = df_new['PURCHASES']
x2 = df_new['PAYMENTS']

plt.figure(figsize=(8,6))
u_labels = np.unique(labels)
for i in u_labels:
    plt.scatter(x1[df_new['cluster_labels'] == i] , x2[df_new['cluster_labels'] == i] , label = i)

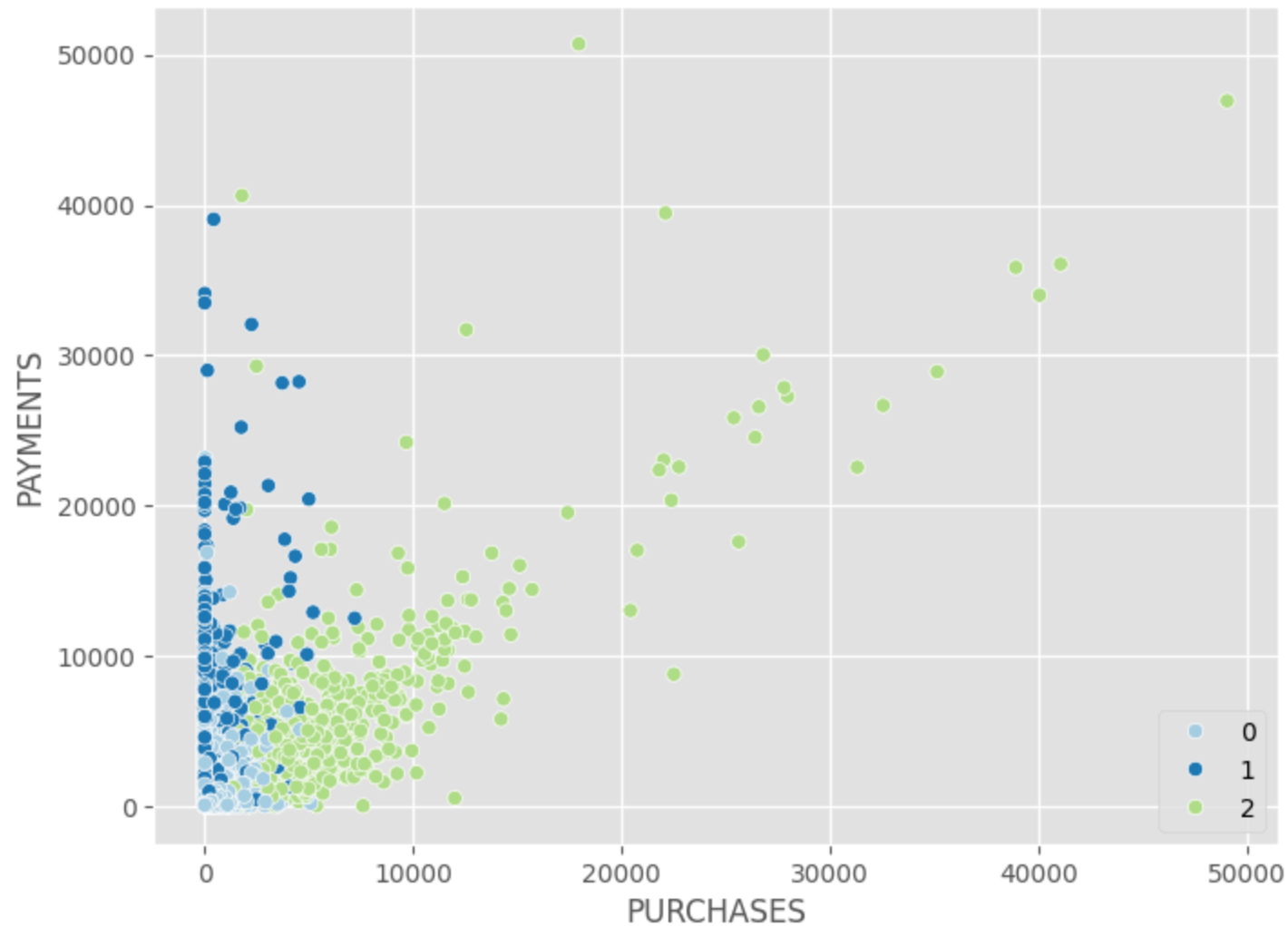
plt.scatter(x1,x2, c=k_means.labels_, cmap='rainbow')
plt.xlabel(x1.name,  fontsize=20)
plt.ylabel(x2.name,  fontsize=20)
plt.title('K-means clustering',fontsize=20)
plt.xticks(fontsize=20)
plt.yticks(fontsize=20)
plt.legend()
plt.show()
```



```
In [ ]: import seaborn as sns

plt.figure(figsize=(8,6))
x_val = 'PURCHASES'
y_val = 'PAYMENTS'
sns.scatterplot(x=x_val, y=y_val, hue='cluster_labels', data=df_new, palette='Paired')

plt.legend(loc='lower right')
plt.show()
```



```
In [ ]: import plotly.express as px
```

```
x_val = 'PURCHASES'
```

```
y_val = 'PAYMENTS'
```

```
z_val = 'BALANCE'
```

```
fig = px.scatter_3d(df_new, x=x_val, y=y_val, z=z_val, color='cluster_labels', labels='cluster_labels')  
fig.show()
```

KMedoids

```
In [ ]: pip install scikit-learn-extra
```

Collecting scikit-learn-extra

Downloading scikit_learn_extra-0.3.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.0 MB)

2.0/2.0 MB 9.8 MB/s eta 0:00:00

Requirement already satisfied: numpy>=1.13.3 in /usr/local/lib/python3.10/dist-packages (from scikit-learn-extra) (1.25.2)

Requirement already satisfied: scipy>=0.19.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn-extra) (1.11.4)

Requirement already satisfied: scikit-learn>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn-extra) (1.2.2)

Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.23.0->scikit-learn-extra) (1.4.0)

Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.23.0->scikit-learn-extra) (3.4.0)

Installing collected packages: scikit-learn-extra

Successfully installed scikit-learn-extra-0.3.0

elbow method

untuk mencari nilai k

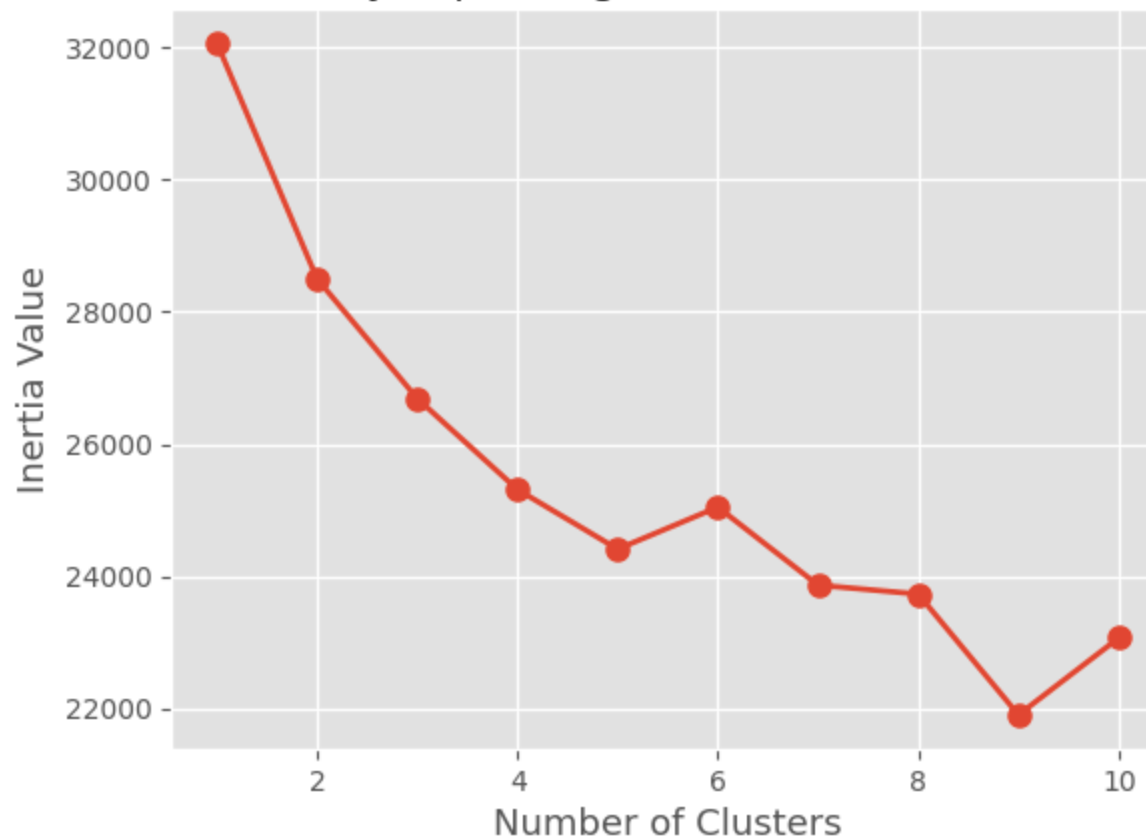
```
In [ ]: from sklearn_extra.cluster import KMedoids

inertia_list = []
for num_clusters in range(1, 11):
    kmedoids_model = KMedoids(n_clusters=num_clusters)
    kmedoids_model.fit(X_new)
    inertia_list.append(kmedoids_model.inertia_)
    print(f"The inertia of {num_clusters} clusters : {kmedoids_model.inertia_}")
```

```
The inertia of 1 clusters : 32074.48996380632
The inertia of 2 clusters : 28502.252083851865
The inertia of 3 clusters : 26690.45199631644
The inertia of 4 clusters : 25312.680447618714
The inertia of 5 clusters : 24410.14906458207
The inertia of 6 clusters : 25045.959805744402
The inertia of 7 clusters : 23865.264137617396
The inertia of 8 clusters : 23728.760889007306
The inertia of 9 clusters : 21901.105869527415
The inertia of 10 clusters : 23064.59573815541
```

```
In [ ]: plt.plot(range(1,11),inertia_list, marker='o', linewidth=2, markersize=8)
plt.xlabel("Number of Clusters", size=13)
plt.ylabel("Inertia Value", size=13)
plt.title("Inertia values vary depending on the number of clusters utilized.")
plt.show()
```

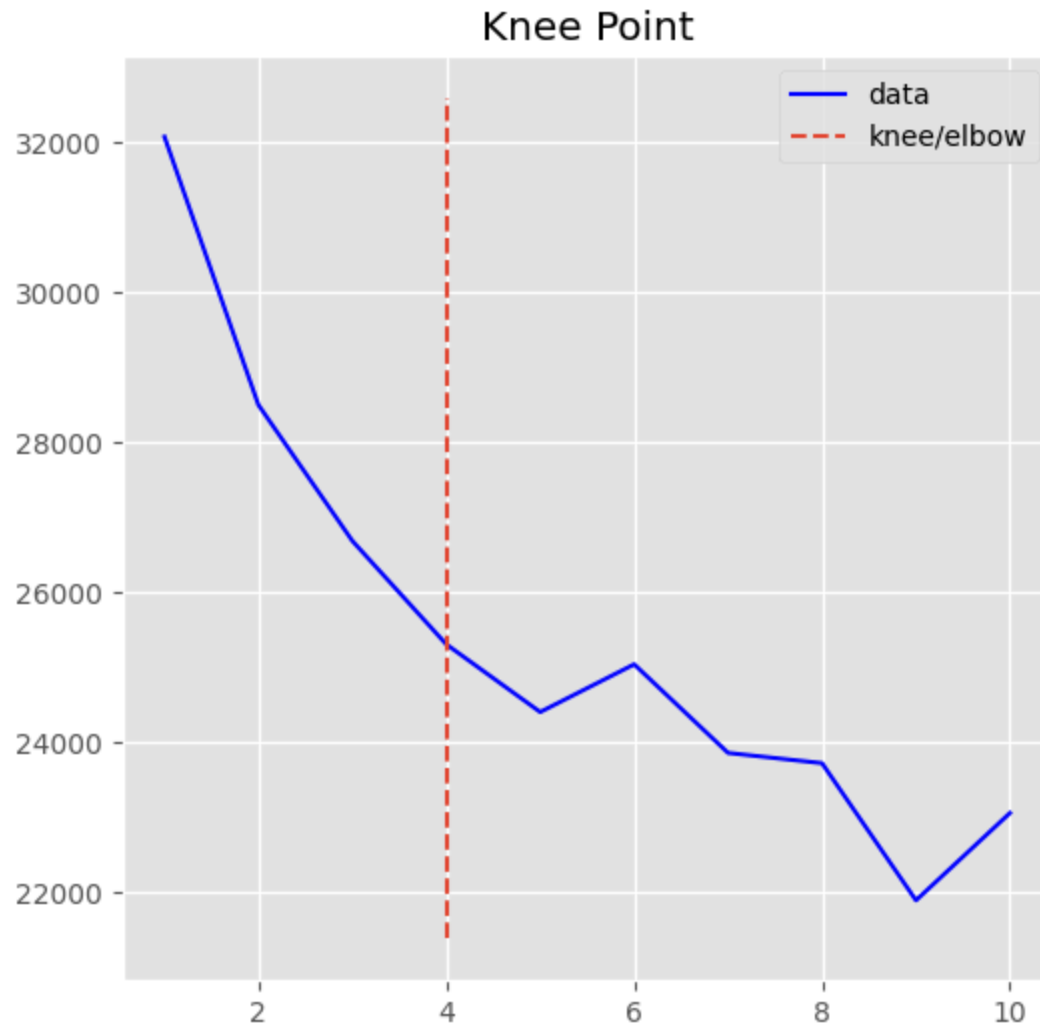
Inertia values vary depending on the number of clusters utilized.



```
In [ ]: from kneed import KneeLocator, DataGenerator
kneedle = KneeLocator(range(1,11),inertia_list, S=1.0, curve='convex', direction='decreasing')
print(round(kneedle.knee, 3))
print(round(kneedle.elbow, 3))
```

4
4

```
In [ ]: plt.style.use('ggplot')  
#kneedle.plot_knee_normalized()  
kneedle.plot_knee()
```



silhouette method

untuk mencari nilai k

```
In [ ]: from sklearn.metrics import silhouette_samples, silhouette_score
sh_list = []
for num_clusters in range(2, 11):
    kmedoids = KMedoids(n_clusters=num_clusters)
    cluster_labels = kmedoids.fit_predict(X_new)

    score = silhouette_score(X_new, cluster_labels)
    sh_list.append(score)
    print("For n_clusters = {}, silhouette score is {}".format(num_clusters, score))
```

```
For n_clusters = 2, silhouette score is 0.1945165537378692)
For n_clusters = 3, silhouette score is 0.16024040458444103)
For n_clusters = 4, silhouette score is 0.13738603460433335)
For n_clusters = 5, silhouette score is 0.14859330798827058)
For n_clusters = 6, silhouette score is 0.07497117420304265)
For n_clusters = 7, silhouette score is 0.051193591100429474)
For n_clusters = 8, silhouette score is 0.03912406632314492)
For n_clusters = 9, silhouette score is 0.08491760468793152)
For n_clusters = 10, silhouette score is 0.03469231089500277)
```

```
In [ ]: plt.plot(range(2,11),sh_list, marker='o', linewidth=2, markersize=8)
plt.xlabel("Number of Clusters", size=13)
plt.ylabel("silhouette score", size=13)
plt.title("Silhouette score vary depending on the number of clusters utilized.")
plt.show()
```

Silhouette score vary depending on the number of clusters utilized.



kmedoids

```
In [ ]: k_medoids = KMedoids(n_clusters = 4, random_state = 42)
k_medoids.fit(X_new)
labels = k_medoids.labels_
df_new['cluster_labels'] = labels
df_new.head()
```

```
Out[35]:
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES
0	40.900749	0.818182	95.40	0.00	95.4	0.000000	
1	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	
2	2495.148862	1.000000	773.17	773.17	0.0	0.000000	
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	
4	817.714335	1.000000	16.00	16.00	0.0	0.000000	

```
In [ ]: centroids = k_medoids.cluster_centers_
centroids
```

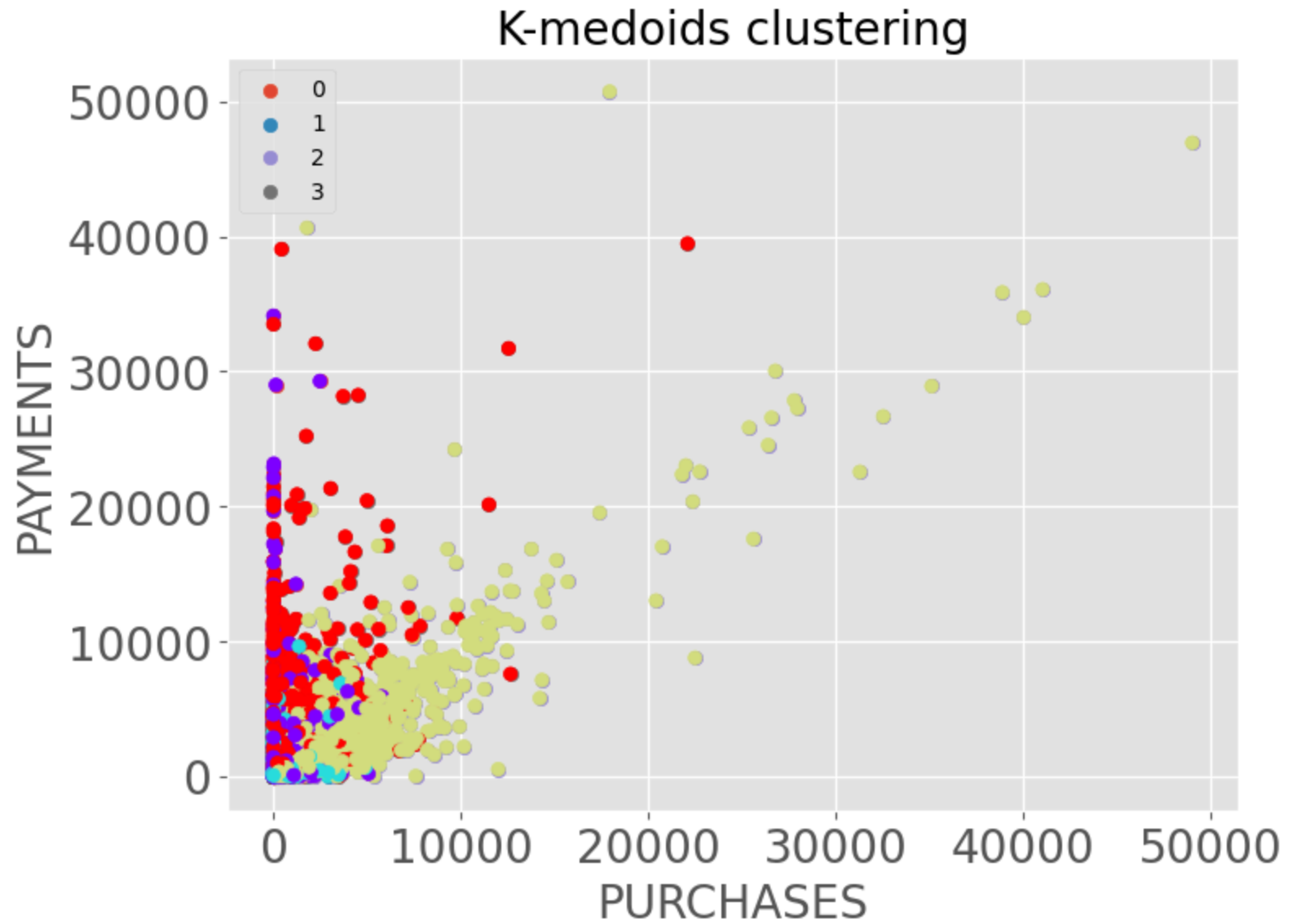
```
Out[36]: array([[ -0.73017008, -1.78447531, -0.15174467,  0.01660604, -0.3893315 ,
-0.46678555, -0.39122513, -0.39931927, -0.49762862, -0.67534886,
-0.47606982, -0.39063931, -0.20456068, -0.35069111, -0.32756103,
 0.50015018,  0.36067954],
[ -0.04321221,  0.51808382, -0.41993839, -0.29307072, -0.45457623,
-0.44947139, -1.01412545, -0.39931927, -0.91699519, -0.25891333,
-0.18299798, -0.51133325, -0.41069279, -0.47801089, -0.17593333,
-0.52555097,  0.36067954],
[ -0.65121444,  0.51808382,  0.24224266,  0.03901242,  0.50039651,
-0.46678555,  1.06221062,  0.15936716,  1.38951716, -0.67534886,
-0.47606982,  0.37375564, -0.27327139,  0.0131038 , -0.2796588 ,
 0.04428414,  0.36067954],
[ 1.16410803,  0.51808382, -0.40096824, -0.32982224, -0.3423    ,
 0.80421891, -0.18359002, -0.39931927, -0.2879466 ,  1.8232743 ,
 0.69621752, -0.350408 ,  0.41383563, -0.14385479,  0.17482124,
-0.52555097,  0.36067954]])
```

Visualization

```
In [ ]: x1 = df_new['PURCHASES']
x2 = df_new['PAYMENTS']

plt.figure(figsize=(8,6))
u_labels = np.unique(labels)
for i in u_labels:
    plt.scatter(x1[df_new['cluster_labels'] == i] , x2[df_new['cluster_labels'] == i] , label = i)

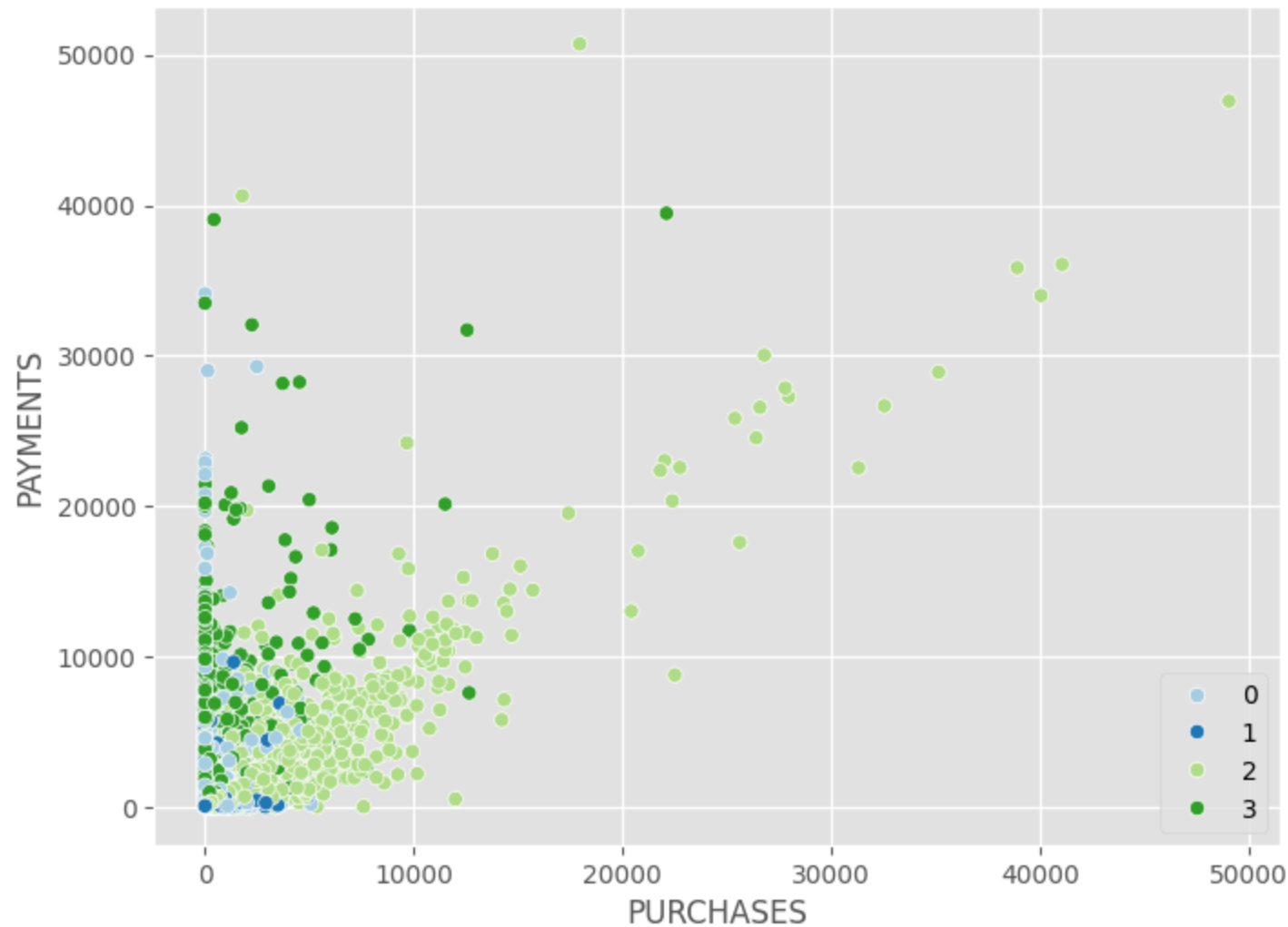
plt.scatter(x1,x2, c=k_medoids.labels_, cmap='rainbow')
plt.xlabel(x1.name,  fontsize=20)
plt.ylabel(x2.name,  fontsize=20)
plt.title('K-medoids clustering',fontsize=20)
plt.xticks(fontsize=20)
plt.yticks(fontsize=20)
plt.legend()
plt.show()
```



```
In [ ]: import seaborn as sns

plt.figure(figsize=(8,6))
x_val = 'PURCHASES'
y_val = 'PAYMENTS'
sns.scatterplot(x=x_val, y=y_val, hue='cluster_labels', data=df_new, palette='Paired')

plt.legend(loc='lower right')
plt.show()
```



```
In [ ]: import plotly.express as px
```

```
x_val = 'PURCHASES'
```

```
y_val = 'PAYMENTS'
```

```
z_val = 'BALANCE'
```

```
fig = px.scatter_3d(df_new, x=x_val, y=y_val, z=z_val, color='cluster_labels', labels='cluster_labels')  
fig.show()
```

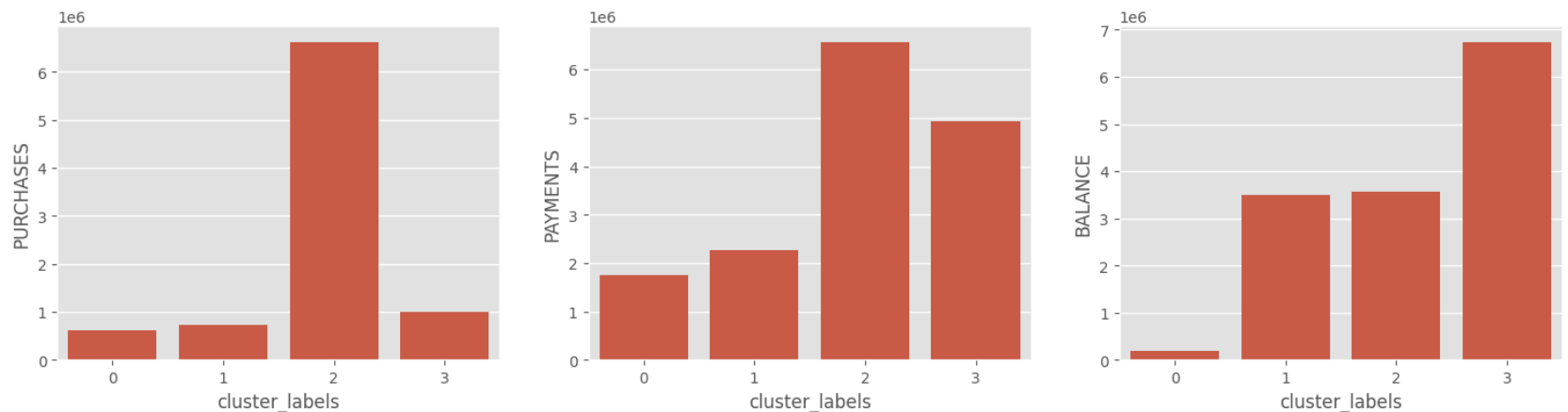
Discussion

```
In [ ]: df_out = df_new.groupby(by = 'cluster_labels').sum()[['PURCHASES', 'PAYMENTS', 'BALANCE']].reset_index()
df_out
```

```
Out[40]:
```

	cluster_labels	PURCHASES	PAYMENTS	BALANCE
0	0	624164.63	1.750357e+06	1.861875e+05
1	1	729117.47	2.268248e+06	3.507307e+06
2	2	6615989.42	6.562254e+06	3.576875e+06
3	3	1009411.74	4.930778e+06	6.731680e+06

```
In [ ]: plt.figure(figsize = (18,4))
plt.subplot(1,3,1)
sns.barplot(x= 'cluster_labels',y = 'PURCHASES', data = df_out)
plt.subplot(1,3,2)
sns.barplot(x= 'cluster_labels',y = 'PAYMENTS', data = df_out)
plt.subplot(1,3,3)
sns.barplot(x= 'cluster_labels',y = 'BALANCE', data = df_out)
plt.show()
```



Cluster 0 : Mempunyai jumlah terendah dalam PURCHASES, PAYMENTS dan BALANCE

Cluster 1 : Mempunyai jumlah sedang dalam PURCHASES, PAYMENTS dan BALANCE

Cluster 2 : Mempunyai jumlah tertinggi dalam PURCHASES, PAYMENTS tetapi jumlah sedang dalam BALANCE

Cluster 3 : Mempunyai jumlah sedang dalam PURCHASES, PAYMENTS tetapi jumlah tertinggi dalam BALANCE

```
In [ ]: #untuk encoding
from sklearn.preprocessing import LabelEncoder

cats = df.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
le = LabelEncoder()
for i in cat_features:
    df[i] = le.fit_transform(df[i])
```

Soal 2: Menggunakan dataset lain, silahkan mengerjakan clustering menggunakan K-Means dan K-medoids

Dataset Air Traffic Passenger Statistics

https://data.sfgov.org/Transportation/Air-Traffic-Passenger-Statistics/rkru-6vcg/about_data